

Hướng dẫn triển khai LearnQuiz

Table of Contents

HƯỚNG DẪN TRIỂN KHAI — LEARNQUIZ

Đưa hệ thống từ máy cá nhân lên Internet: **Backend + PostgreSQL trên Render**, **Frontend trên Vercel**. Toàn bộ dùng gói miễn phí, không cần thẻ tín dụng. Hướng dẫn theo từng cú nhấp chuột — làm theo đúng thứ tự, không bỏ bước.

Thời gian dự kiến: 25–35 phút cho lần triển khai đầu tiên (chủ yếu là thời gian Render dựng máy chủ).

Hệ thống thật đang chạy (đã deploy):

Thành phần	URL
Frontend (Vercel)	https://final-project-js-ten.vercel.app/
Backend API (Render)	https://learnquiz-api.onrender.com/ — kiểm tra nhanh: <code>/health</code>
Mã nguồn (GitHub)	https://github.com/tamthientinvu-coder/FinalProjectJS

⚠️ QUAN TRỌNG NHẤT — CSDL Render Free hết hạn và bị xóa tự động

Trang Info của `learnquiz-db` trên Render Dashboard ghi rõ: “Your database will expire on **27/09/2026**. The database will be deleted unless you upgrade to a paid compute plan.” — đây **không phải** hiện tượng “ngủ” tạm thời như web service Free, mà là **xóa vĩnh viễn toàn bộ dữ liệu** đúng ~30 ngày sau khi tạo, trừ khi nâng cấp lên gói trả phí.

Cha kiểm tra ngay: nếu ngày bảo vệ đồ án rơi **sau 27/09/2026**, hệ thống sẽ sập hoàn toàn (báo lỗi kết nối CSDL) nếu không xử lý trước. Ba lựa chọn, làm một trong ba **trước ngày hết hạn**:

1. **Nâng cấp `learnquiz-db` lên gói trả phí thấp nhất của Render** (Dashboard — `learnquiz-db` — banner cảnh báo — “*upgrade to a paid compute plan*”) — dữ liệu giữ nguyên, không gián đoạn, nhưng bắt đầu tốn phí hàng tháng.
2. **Tạo CSDL Free mới ngay trước ngày bảo vệ** (vài ngày trước là đủ) rồi trở `DATABASE_URL` trên `learnquiz-api` sang CSDL mới, chạy lại `migrate + seed` — miễn phí nhưng phải nhớ làm lại đúng lịch mỗi ~30 ngày.
3. Nếu chỉ demo bằng Docker cục bộ (Bước 7) trong buổi bảo vệ thì không bị ảnh hưởng, vì Docker dùng CSDL riêng trên máy, không liên quan tới CSDL Render.

Xem lại ngày hết hạn thật trên Render Dashboard trước khi quyết định — ngày 27/09/2026 là ngày quan sát được tại thời điểm viết tài liệu này, có thể lệch nếu CSDL được tạo lại.

Mục lục

0. Chuẩn bị
 1. Đẩy mã nguồn lên GitHub
 2. Lấy API key Google Gemini
 3. Backend + Cơ sở dữ liệu trên Render
 4. Frontend trên Vercel
 5. Nói hai đầu lại — bước hay bị quên nhất
 6. Kiểm tra toàn diện sau khi deploy
 7. Chạy trọn bộ bằng Docker (phương án dự phòng)
 8. Tích hợp liên tục (CI)
 9. Quay lui khi deploy hỏng
 10. Checklist trước buổi bảo vệ
 11. Bảng tra lỗi thường gặp
 12. Giám sát & thông báo sau khi deploy
 13. Quản lý dữ liệu bằng Prisma Studio từ máy tính
-

0. Chuẩn bị

Việc cần làm	Ghi chú
Tài khoản GitHub	Miễn phí, dùng để lưu mã nguồn và cho Render/Vercel đọc
Tài khoản render.com	Đăng nhập bằng Sign in with GitHub — không cần tạo mật khẩu riêng
Tài khoản vercel.com	Đăng nhập bằng Continue with GitHub
Tài khoản aistudio.google.com	Đăng nhập bằng Gmail bất kỳ, dùng để lấy API key Gemini
Docker Desktop (tuỳ chọn)	Chỉ cần nếu muốn có phương án dự phòng offline ở Bước 7

Nếu đã có sẵn repository trên GitHub và đã có API key Gemini, **bỏ qua Bước 1 và 2**, đi thẳng vào Bước 3.

1. Đẩy mã nguồn lên GitHub (bỏ qua nếu đã làm)

1.1. Kiểm tra .env chưa từng bị commit

Việc này làm **trước tiên**, kể cả khi repo đã tồn tại từ lâu:

```
cd FinalProject
git log --all --full-history -- "*/.env" ".env"
```

Kết quả phải **rỗng** (không in ra dòng nào). Nếu có kết quả nghĩa là secret đã lộ ra GitHub — phải gỡ khỏi lịch sử git (`git filter-repo` hoặc tạo repo mới) và **đổi toàn bộ secret liên quan** (JWT secret, mật khẩu DB, API key), vì bất cứ thứ gì từng lên GitHub — kể cả sau khi xóa — đều phải coi như đã công khai.

Tập `.gitignore` ở gốc dự án đã chặn sẵn `.env`, `.env.local`, `.env.*.local` — không cần sửa gì thêm.

Repo của đồ án này đã tồn tại sẵn tại <https://github.com/tamthientinvu-coder/FinalProjectJS>, đã push và đã được Render + Vercel đọc để deploy. Mục 1.2–1.3 dưới đây là các bước **đã làm xong** cho repo này — giữ lại làm tài liệu tham khảo nếu sau này cần tạo lại từ đầu (máy khác, tài khoản khác...). Có thể bỏ qua, đi thẳng Bước 2.

1.2. Tạo repository trên GitHub

1. Đăng nhập github.com → góc trên bên phải, bấm dấu + → **New repository**
2. Đặt tên, ví dụ `FinalProjectJS` → chọn **Private** (khuyến nghị trong lúc chưa bảo vệ xong) hoặc **Public**
3. **Không** tick “Add a README file” (dự án đã có sẵn) → bấm **Create repository**
4. GitHub hiện ra một trang có sẵn các dòng lệnh — copy đoạn dưới mục “...or push an existing repository from the command line”

1.3. Đẩy mã nguồn

Trong thư mục gốc dự án (`FinalProject/`):

```
git init # nếu chưa từng git
git add .
git commit -m "Initial commit - LearnQuiz"
git branch -M main
git remote add origin
https://github.com/tamthientinvu-coder/FinalProjectJS.git
git push -u origin main
```

(Tài khoản GitHub của đồ án: **tamthientinvu-coder** — email đăng ký `tamthientinvu@gmail.com`.) Sau khi chạy xong, tải lại trang GitHub — phải thấy đủ hai thư mục `backend/`, `frontend/` và tệp `render.yaml` ở gốc.

Kiểm tra lại lần nữa trên GitHub: vào tab **Code**, gõ `.env` vào ô tìm kiếm của repo (phím `t` để bật tìm tệp nhanh) — không được ra kết quả nào ngoài `.env.example`.

2. Lấy API key Google Gemini

1. Vào aistudio.google.com/apikey → đăng nhập Gmail
2. Bấm **Create API key**

3. Chọn **Create API key in new project** (nếu chưa có project Google Cloud nào)
4. Copy chuỗi key hiện ra (dạng AIzaSy...) — **dán ngay vào một chỗ ghi chú tạm**, màn hình này không hiện lại đầy đủ key sau khi đóng
5. Gói miễn phí (Free tier) của Gemini đã đủ dùng cho việc demo — không cần thẻ tín dụng

Key này sẽ dùng ở Bước 3, mục biến môi trường GEMINI_API_KEY.

3. Backend + Cơ sở dữ liệu trên Render

Cách A — Dùng Blueprint (khuyến nghị, nhanh nhất)

Dự án đã có sẵn `render.yaml` ở thư mục gốc, khai báo đủ cả CSDL lẫn API — Render đọc tệp này và tự dựng mọi thứ.

1. Đăng nhập dashboard.render.com
2. Góc trên bên phải — bấm nút tím **New** — chọn **Blueprint** trong danh sách sổ xuống
3. Nếu đây là lần đầu Render truy cập GitHub: bấm **Configure account** — chọn **Only select repositories** — tick vào repo FinalProjectJS (tài khoản tamthientinvu-coder) — **Install**
4. Quay lại màn hình Blueprint, chọn repository FinalProjectJS trong danh sách — bấm **Connect**
5. Render tự đọc `render.yaml` và hiện ra bản xem trước gồm **2 tài nguyên**: database `learnquiz-db` và web service `learnquiz-api` (tên hai tài nguyên này do `render.yaml` đặt, khác với tên repo GitHub)
6. Đặt tên cho Blueprint (không quan trọng, ví dụ `learnquiz`) — kéo xuống dưới
7. Render sẽ hỏi giá trị cho các biến có `sync: false` trong `render.yaml` — đúng 2 ô:
 - FE_URL — tạm điền `http://localhost:5173` (sẽ sửa lại ở Bước 5, vì lúc này Vercel chưa deploy)
 - GEMINI_API_KEY — dán key vừa lấy ở Bước 2 (Các biến còn lại như `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` có `generateValue: true` nên Render tự sinh chuỗi ngẫu nhiên, không cần điền.)
8. Bấm **Apply**
9. Render chuyển sang màn hình build log — chờ khoảng **3–5 phút**. Bốn việc diễn ra tuần tự, theo dõi được trong log:
 - Tạo PostgreSQL (trạng thái chuyển từ *Creating* → *Available*)
 - `npm ci --include=dev && npx prisma generate && npx prisma migrate deploy && npm run seed && npm run build` cho web service — cờ `--include=dev` bắt buộc phải có vì lệnh `npm run seed` chạy bằng `ts-node`, nằm trong `devDependencies` mà Render mặc định bỏ qua ở môi trường `production`
 - Nạp dữ liệu mẫu (`npm run seed`) — script này **tự xóa dữ liệu khoá học/quiz cũ rồi tạo lại từ đầu**, còn tài khoản thì `upsert` (giữ nguyên nếu đã tồn tại) — xem thêm mục “Nạp dữ liệu mẫu” bên dưới

- Container khởi động, Render tự gọi `healthCheckPath: /health` để xác nhận service sống

10. Khi cả hai thẻ tài nguyên chuyển sang chấm tròn **xanh lá**, việc dựng hạ tầng đã xong

Copy URL của web service (dạng `https://learnquiz-api.onrender.com`, hiện ngay dưới tên service) — sẽ dùng ở Bước 4 và 5.

Cách B — Tạo tay từng bước

Dùng khi muốn kiểm soát từng biến, hoặc khi `render.yaml` không áp dụng được (ví dụ tài khoản Render dạng tổ chức có giới hạn Blueprint).

Bấm để xem hướng dẫn tạo tay

B1. Tạo cơ sở dữ liệu

1. Dashboard → **New** → **PostgreSQL**
2. Điền:

Trường	Giá trị
Name	learnquiz-db
Database	learnquiz_db
User	learnquiz
Region	Singapore (<i>gần Việt Nam nhất, giảm độ trễ</i>)
PostgreSQL Version	để mặc định — Render tự chọn bản mới nhất tại thời điểm tạo (thực tế CSDL của đồ án này ra bản 18 , khác với bản 16 dùng ở Docker cục bộ; Prisma tương thích cả hai nên không ảnh hưởng)
Plan	Free

3. Bấm **Create Database**
4. Chờ trạng thái chuyển thành **Available** (khoảng 1 phút)
5. Trong trang chi tiết database, cuộn xuống mục **Connections** → copy chuỗi **Internal Database URL** (không phải External — xem lý do ở Bước 11)

B2. Tạo web service

1. Dashboard → **New** → **Web Service**
2. Chọn repository `FinalProjectJS` → **Connect**
3. Điền:

Trường	Giá trị
Name	learnquiz-api
Region	Singapore
Branch	main
Root Directory	backend
Runtime	Node

Trường	Giá trị
Build Command	<code>npm ci --include=dev && npx prisma generate && npx prisma migrate deploy && npm run seed && npm run build</code>
Start Command	<code>node dist/index.js</code>
Plan	Free
4. Kéo xuống mục Health Check Path (trong phần <i>Advanced</i>) → điền <code>/health</code>	
5. Mục Environment Variables → bấm Add Environment Variable cho từng dòng:	
Key	Value
NODE_ENV	production
DATABASE_URL	Internal Database URL vừa copy ở B1
JWT_ACCESS_SECRET	bấm nút Generate bên cạnh ô (Render tự sinh chuỗi ngẫu nhiên)
JWT_REFRESH_SECRET	bấm Generate — phải là chuỗi khác với JWT_ACCESS_SECRET
JWT_ACCESS_EXPIRES	15m
JWT_REFRESH_EXPIRES	7d
FE_URL	tạm <code>http://localhost:5173</code> , sửa lại ở Bước 5
GEMINI_API_KEY	key lấy ở Bước 2
GEMINI_MODEL	gemini-2.0-flash
6. Bấm Create Web Service ở cuối trang	
7. Theo dõi tab Logs — chờ tới khi thấy dòng server khởi động thành công và chấm trạng thái chuyển xanh lá	

Vì sao `migrate deploy` nằm ở `Build Command` chứ không ở `Start Command`?

Build chạy **một lần mỗi lần deploy**. Start chạy **mỗi lần container khởi động lại** — mà gói Free của Render cho service ngủ rồi tự dậy liên tục (xem Bước 6). Đặt migration ở Start là mỗi lần dậy lại chạy migration một lần, vừa chậm vừa dễ đụng nhau nếu có nhiều container cùng khởi động.

Lưu ý: dùng `migrate deploy` chứ **không** dùng `migrate dev`. `migrate dev` có thể xóa và tạo lại cơ sở dữ liệu — tuyệt đối không đưa vào production.

Nạp dữ liệu mẫu — hiện đã tự động chạy mỗi lần deploy

`render.yaml` đã có `npm run seed` ngay trong `buildCommand`, nên **không cần vào Shell chạy tay** như trước nữa — cứ mỗi lần đẩy commit mới lên main (Render tự deploy lại), dữ liệu mẫu tự nạp lại.

Script `prisma/seed.ts` viết theo nguyên tắc **idempotent** (chạy lại nhiều lần không lỗi): - **Tài khoản** (`admin@learnquiz.vn`, `instructor@learnquiz.vn`, ...) dùng `upsert` — đã

tồn tại thì giữ nguyên, không tạo trùng. - **Khoá học, bài học, quiz, lượt đăng ký, bài nộp bị xoá sạch rồi tạo lại từ đầu** mỗi lần seed chạy.

Hệ quả cần biết: nếu trong lúc demo có tạo khoá học mới hay nộp quiz thật, dữ liệu đó sẽ **mất** ở lần deploy kế tiếp (mỗi lần push code lên main). Đây là đánh đổi hợp lý cho một hệ thống demo — luôn có dữ liệu mẫu sạch để trình bày — nhưng không phù hợp nếu sau này đưa vào dùng thật với người dùng thật.

Nếu cần seed lại thủ công mà không muốn deploy lại (ví dụ nghi dữ liệu bị lỗi giữa chừng): Dashboard Render → mở service learnquiz-api → tab **Shell** → gõ `npm run seed`.

Kiểm tra

Mở tab trình duyệt mới, dán:

`https://learnquiz-api.onrender.com/health`

(thay bằng URL thật của service) — phải thấy:

```
{ "status": "ok", "db": "up", "uptime": 12.34 }
```

Nếu db không phải "up", xem mục [Bảng tra lỗi](#).

4. Frontend trên Vercel

1. Đăng nhập vercel.com/new
2. Nếu chưa cấp quyền GitHub: bấm **Add GitHub Account** → chọn repo `FinalProjectJS` → **Install**
3. Trong danh sách repository hiện ra, tìm `FinalProjectJS` → bấm **Import**
4. Màn hình **Configure Project** hiện ra:

Trường	Giá trị
Project Name	tùy ý — Vercel tự đặt theo tên repo (<code>FinalProjectJS</code>) và thêm hậu tố ngẫu nhiên nếu trùng người khác đã dùng, nên đồ án này ra domain <code>final-project-js-ten.vercel.app</code>
Framework Preset	Vite (Vercel thường tự nhận diện đúng)
Root Directory	bấm Edit bên cạnh → gõ <code>frontend</code> → Continue
Build Command	để nguyên mặc định <code>npm run build</code>
Output Directory	để nguyên mặc định <code>dist</code>
Install Command	để nguyên mặc định
5. Bấm mũi tên mở rộng mục Environment Variables → thêm:	
Key	Value
<code>VITE_API_URL</code>	<code>https://learnquiz-</code>

Key	Value
	<code>api.onrender.com/api/v1</code> (URL Render ở Bước 3, nhớ thêm đuôi <code>/api/v1</code>)
6.	Bấm Deploy
7.	Vercel build trong khoảng 30–60 giây, xong hiện màn hình pháo giấy 🎆 kèm ảnh chụp trang web và nút Continue to Dashboard
8.	Domain chính thức hiện ở đầu trang dashboard, dạng <code>https://final-project-js-ten.vercel.app</code> — copy lại, dùng ở Bước 5

Hai điều dễ sai ở bước này

Biên VITE_* được nhúng lúc BUILD, không đọc lúc chạy. Đổi `VITE_API_URL` xong phải vào tab **Deployments** — bấm ... ở bản mới nhất — **Redeploy**, chứ không phải chỉ chờ hay bấm restart — restart không tồn tại với Vercel vì đây là static hosting, không có tiến trình để restart. Cũng chính vì biên build-time bị nhúng thẳng vào file JS công khai, **tuyệt đối không đặt GEMINI_API_KEY ở đây** dù chỉ để thử — bất kỳ ai mở DevTools — tab Sources đều đọc được.

React Router cần rewrite. Không có nó thì gõ thẳng `https://.../courses/5` vào thanh địa chỉ sẽ ra lỗi 404 của Vercel, vì trên đĩa không tồn tại file nào tên `courses/5` — chỉ có `index.html` và JS bundle, còn việc hiển thị đúng trang là do React Router xử lý *sau khi* đã tải xong ở trình duyệt. File `frontend/vercel.json` đã khai báo sẵn, không cần chỉnh:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Đo hiệu năng thực tế bằng Vercel Speed Insights

Frontend đã cài `@vercel/speed-insights` và gắn sẵn `<SpeedInsights />` trong `main.tsx` — không cần thêm biến môi trường, không cần bật gì trên Vercel. Khi frontend chạy trên hạ tầng Vercel, chỉ số Core Web Vitals (LCP, CLS, INP...) tự động xuất hiện ở tab **Speed Insights** của project sau vài phút có lượt truy cập thật. Khi chạy `npm run dev` ở máy cá nhân hoặc trong Docker, thư viện tự nhận diện không phải môi trường Vercel và **không gửi dữ liệu đi đâu cả** — an toàn, không ảnh hưởng lúc phát triển.

5. Nối hai đầu lại — bước hay bị quên nhất

1. Quay lại dashboard.render.com — mở service `learnquiz-api`
2. Chọn tab **Environment** ở menu ngang
3. Tìm dòng `FE_URL` — bấm vào ô giá trị — sửa thành:
`https://final-project-js-ten.vercel.app`

(dán đúng domain Vercel vừa cập ở Bước 4, **không** có dấu / ở cuối)

4. Bấm **Save Changes** ở góc dưới bên phải
5. Render tự động deploy lại (không cần build lại toàn bộ, chỉ khởi động lại container với biên môi trường mới) — chờ chấm trạng thái chuyển lại màu xanh lá, khoảng 30–60 giây

Thiếu bước này thì frontend gọi API sẽ dính lỗi CORS: mở trang web, bấm F12 mở DevTools — tab Console sẽ thấy dòng đỏ *“has been blocked by CORS policy”* — còn phía Render Logs thì **không hề báo lỗi gì**, vì request đã bị chính trình duyệt chặn lại trước khi rời máy người dùng. Đây là nguyên nhân số một khiến người mới triển khai tưởng backend bị hỏng trong khi thực ra backend hoàn toàn bình thường.

6. Kiểm tra toàn diện sau khi deploy

Làm tuần tự, đừng bỏ qua bước nào — mỗi bước xác nhận một tầng khác nhau của hệ thống:

1. **Tầng mạng/CSDL:** mở `https://learnquiz-api.onrender.com/health` — phải ra `{ "status": "ok", "db": "up" }`
2. **Tầng CORS:** mở `https://final-project-js-ten.vercel.app`, bấm F12 — tab **Console** — tải lại trang (F5) — không được có dòng đỏ nào nhắc tới CORS
3. **Tầng dữ liệu mẫu:** đăng nhập bằng tài khoản demo (xem README.md mục tài khoản mẫu) — phải vào được, thấy danh sách khoá học
4. **Tầng nghiệp vụ:** mở một khoá học, làm thử một bài quiz, nộp bài — phải nhận được điểm và không thấy trường `isCorrect` nào bị lộ ra trước khi nộp (kiểm tra bằng tab **Network** của DevTools, xem response của API lấy câu hỏi)
5. **Tầng AI (nếu bật):** vào trang soạn quiz với vai trò giảng viên/quản trị — bấm **Sinh câu hỏi bằng AI** — phải nhận được câu hỏi soạn sẵn trong vài giây; nếu báo lỗi 429 nghĩa là hết hạn mức Gemini miễn phí, không phải lỗi hệ thống

Chỉ khi cả 5 bước trên đều đạt thì mới coi là triển khai thành công.

7. Chạy trọn bộ bằng Docker (phương án dự phòng)

Dùng khi mạng tại nơi bảo vệ đồ án không ổn định, hoặc muốn trình diễn mà không phụ thuộc Render/Vercel còn sống hay không. Chỉ cần máy tính cá nhân đã cài Docker Desktop, **không cần Internet** sau khi đã tải image lần đầu.

1. Mở Terminal (hoặc PowerShell trên Windows) tại thư mục gốc `FinalProject/`
2. Chạy:
`docker compose -f docker-compose.full.yml up --build`
3. Lần đầu chạy mất khoảng 2–4 phút để build hai image (backend, frontend) và tải image PostgreSQL. Các lần sau chạy lại chỉ mất vài giây nếu không đổi mã nguồn.
4. Chờ tới khi thấy log đứng yên với dòng dạng `Server is running on port 3000` — nghĩa là cả ba container đã chạy: `learnquiz_db_full`, `learnquiz_api`, `learnquiz_web`

Thành phần	Địa chỉ
Frontend (nginx)	<code>http://localhost:8080</code>
Backend API	<code>http://localhost:3000</code>

Thành phần	Địa chỉ
PostgreSQL	localhost:5432

5. **Nạp dữ liệu mẫu** — mở một cửa sổ Terminal khác (để cửa sổ đầu tiên tiếp tục chạy), gõ:

```
docker compose -f docker-compose.full.yml exec backend npm run seed
```

6. Mở trình duyệt tại `http://localhost:8080`, đăng nhập thử như Bước 6

Dừng hệ thống: quay lại cửa sổ Terminal đầu tiên, bấm `Ctrl + C`, rồi:

```
docker compose -f docker-compose.full.yml down
```

(thêm `cờ -v` ở cuối nếu muốn xóa luôn dữ liệu đã nạp, để lần sau chạy lại từ đầu)

Vì sao image an toàn hơn chạy `npm run dev` trực tiếp: cả hai image đều dùng **build hai giai đoạn** (multi-stage) — giai đoạn đầu biên dịch TypeScript, giai đoạn sau chỉ giữ lại kết quả đã build. Image backend cuối cùng không chứa mã nguồn TypeScript, không chứa `devDependencies`, và **chạy bằng user node chứ không phải root** — nếu có lỗi hổng trong container, kẻ tấn công cũng không có quyền root ngay từ đầu.

8. Tích hợp liên tục (CI)

`.github/workflows/ci.yml` chạy tự động mỗi lần push lên main và mỗi khi mở Pull Request — xem kết quả tại tab **Actions** trên GitHub:

Job	Nội dung
backend	<code>prisma validate</code> — <code>typecheck</code> — <code>npm test</code> (318 phép kiểm) — <code>build</code>
frontend	<code>typecheck</code> — <code>build production</code>
security	Chặn nếu <code>.env</code> từng bị commit · <code>npm audit</code> cả hai dự án

Bộ kiểm thử dùng Prisma giả lập trong bộ nhớ nên **CI không cần dựng cơ sở dữ liệu thật** — chạy xong trong khoảng một phút. Nếu một job báo đỏ (❌) trên GitHub, bấm vào job đó để xem log chi tiết trước khi push tiếp.

9. Quay lui khi deploy hỏng

Vercel: 1. Dashboard project — tab **Deployments** 2. Tìm bản chạy tốt gần nhất (có chấm xanh, thường là bản trước bản vừa hỏng) 3. Bấm `...` ở cuối dòng — **Promote to Production** 4. Có hiệu lực tức thì, **không cần build lại** — vì Vercel giữ lại mọi bản build cũ

Render: 1. Mở service — tab **Events** 2. Tìm deploy cũ đã chạy ổn định 3. Bấm **Rollback to this deploy** 4. Render dựng lại container từ đúng commit đó — mất khoảng 1–2 phút, không phải tức thì như Vercel

Bảng git (khi cần sửa hẵn mã nguồn thay vì chỉ quay lại bản cũ):

```
git log --oneline -10
git revert <mã-commit-gây-lỗi>
git push
```

Dùng revert chứ không reset --hard trên nhánh chung: revert tạo ra một commit **mới** đảo ngược thay đổi lỗi, lịch sử của cả nhóm vẫn nguyên vẹn; reset --hard xoá lịch sử và sẽ làm hỏng bản sao của người khác nếu họ đã pull commit cũ.

10. Checklist trước buổi bảo vệ

Hạ tầng

- ☐ **learnquiz-db chưa quá ngày hết hạn ghi trên Render Dashboard** (xem cảnh báo đầu tài liệu) — nếu ngày bảo vệ sau ngày hết hạn, phải nâng cấp hoặc tạo CSDL mới trước
- ☐ /health trả {"status": "ok", "db": "up"}
- ☐ Frontend tải được, không có lỗi CORS trong Console
- ☐ Migration đã áp dụng, seed đã chạy, đăng nhập được bằng tài khoản demo
- ☐ Không biến môi trường nào còn trở localhost

Bảo mật

- ☐ git log --all --full-history -- "*/.env" cho kết quả rỗng
- ☐ npm audit không còn lỗi hồng critical/high
- ☐ helmet() đã bật (kiểm tra header x-content-type-options: nosniff bằng tab Network)
- ☐ Rate limit có ở /auth/* và /ai/*
- ☐ Không response nào chứa password hay refreshToken
- ☐ NODE_ENV=production trên Render

Chất lượng mã

- ☐ npm run build sạch lỗi ở cả hai dự án
- ☐ npm test đạt 100% (318/318)
- ☐ Không còn console.log rải rác
- ☐ Mọi route async đều try/catch + next(err)

Chống sự cố khi demo

- ☐ Ping /health **trước buổi bảo vệ 15-30 phút** để service Render tỉnh dậy (gói Free ngủ sau 15 phút không dùng, lần gọi đầu chờ tới ~30-50 giây)
- ☐ Chuẩn bị sẵn ảnh chụp màn hình kết quả AI, phòng khi Gemini hết hạn mức miễn phí đúng lúc demo
- ☐ Mở sẵn tab /health và tab Network để chứng minh isCorrect không rò rỉ

- ❑ Cài sẵn Docker Desktop và thử chạy `docker compose -f docker-compose.full.yml up --build` **ít nhất một lần trước ngày bảo vệ** — đừng để lần chạy đầu tiên là ngay tại hội đồng
- ❑ Có bản in hoặc file PDF của báo cáo phòng khi máy chiếu không kết nối được Internet

11. Bảng tra lỗi thường gặp

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
blocked by CORS policy trong Console	FE_URL trên Render sai hoặc còn dấu / cuối	Vào Render → Environment → sửa FE_URL → Save, chờ deploy lại
Vào thẳng /courses/5 ra 404	Thiếu rewrite của SPA	Kiểm tra <code>frontend/vercel.json</code> có đúng nội dung như Bước 4
Can't reach database server trong log Render	Dùng External Database URL thay vì Internal	Đổi DATABASE_URL sang Internal Database URL (Database → tab Connections)
Mọi API trả lỗi 500, /health báo db không phải up, đột ngột không rõ lý do	CSDL Free đã tới ngày hết hạn ~30 ngày và bị Render xóa	Kiểm tra banner cảnh báo trên trang <code>learnquiz-db</code> — xem mục cảnh báo đầu tài liệu để nâng cấp hoặc tạo CSDL mới
relation "users" does not exist	Chưa chạy migration	Kiểm tra Build Command có <code>npm prisma migrate deploy</code> ; xem lại tab Logs xem bước này có chạy không
Request đầu tiên chờ ~30–50 giây	Service Free của Render đang ngủ (sau 15 phút không có request)	Ping /health trước khi demo, hoặc set thêm dịch vụ ping ngoài (<code>uptimerobot.com</code> , tùy chọn)
Nút “Sinh câu hỏi bằng AI” bị mờ/không bấm được	Thiếu GEMINI_API_KEY trên Render	Thêm biến ở tab Environment rồi Save (Render tự deploy lại)
AI báo lỗi 429	Hết hạn mức miễn phí Gemini trong ngày	Chờ reset hạn mức hoặc đổi key khác; hệ thống vẫn chạy bình thường, chỉ tính năng AI tạm ngưng
Frontend vẫn gọi localhost:3000 dù đã sửa VITE_API_URL	Đổi biến nhưng chưa Redeploy	Vercel → Deployments → ... — Redeploy , không phải chỉ chờ hay restart
Build Render báo lỗi prisma generate không tìm thấy	Thiếu bước <code>npm prisma generate</code> trong Build	Xoá cache: Render → Settings — Clear build cache &

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
engine	Command, hoặc cache Render cũ	deploy
Deploy Vercel báo Root Directory không tìm thấy package.json	Chưa đổi Root Directory thành frontend	Vercel → Settings → General → Root Directory → sửa thành frontend → Save → Redeploy
Build báo lỗi thiếu ts-node khi chạy npm run seed	Build Command thiếu cờ --include=dev — Render mặc định bỏ devDependencies ở production	Sửa Build Command thành npm ci --include=dev && ... như Bước 3
Sau khi deploy lại, khoá học/quiz vừa demo biến mất	npm run seed tự chạy lại mỗi lần deploy, luôn xoá và tạo lại dữ liệu khoá học	Đây là hành vi cố ý của render.yaml hiện tại, không phải lỗi — xem mục “Nạp dữ liệu mẫu” ở Bước 3

12. Giám sát & thông báo sau khi deploy

12.1. Vercel Web Analytics — đã cài

frontend đã cài @vercel/analytics và gắn <Analytics /> trong main.tsx (cùng chỗ với <SpeedInsights />) — không cần thêm biến môi trường. Khác với Speed Insights (đo hiệu năng tải trang), Web Analytics đếm **lượt xem trang và người dùng** (pageview, visitor, trang được xem nhiều nhất).

Để xem số liệu: Vercel Dashboard → chọn project → tab **Analytics**. Lần đầu vào tab này, Vercel có thể yêu cầu bấm **Enable** một lần (sản phẩm tách riêng khỏi việc deploy) — sau khi bật, số liệu xuất hiện dần theo lượt truy cập thật, không hồi tố dữ liệu cũ.

12.2. Preview Deployments — đã bật sẵn, không cần cài

Đây là hành vi **mặc định** của Vercel khi project được kết nối qua GitHub — không có nút “cài đặt” riêng. Cơ chế: mỗi khi có commit mới trên một nhánh **khác** main (hoặc mở Pull Request), Vercel tự build và cấp một URL xem trước độc lập, không ảnh hưởng tới bản production đang chạy tại final-project-js-ten.vercel.app.

Cách kiểm tra đang hoạt động:

```
git checkout -b thu-nghiem-preview
git commit --allow-empty -m "test: kiểm tra preview deployment"
git push -u origin thu-nghiem-preview
```

Mở repo trên GitHub → tạo Pull Request từ nhánh này vào main — bot **Vercel** sẽ tự bình luận vào PR kèm link dạng https://final-project-js-ten-git-thu-nghiem-preview-...vercel.app. Kiểm tra xong thì xoá Pull Request và nhánh, không cần merge.

(Cũng có thể xem trực tiếp không cần tạo PR: Vercel Dashboard → tab Deployments → mọi bản build từ nhánh khác main đều gắn nhãn “Preview” thay vì “Production”).

12.3. Uptime Ping — đã cài bằng GitHub Actions

Thêm workflow `.github/workflows/uptime-ping.yml`, chạy lịch **mỗi 10 phút**, gọi `https://learnquiz-api.onrender.com/health` và kiểm tra cả mã HTTP lẫn nội dung JSON (`status: ok, db: up`). Repo này ở chế độ **public** nên chạy lịch không tốn giới hạn phút chạy miễn phí của private repo.

Hai tác dụng cùng lúc: - **Giữ ấm** service Render free tier — ping đều đặn dưới 15 phút nên service không kịp vào trạng thái ngủ, gần như xóa bỏ độ trễ khởi động lạnh khi giảm viên hoặc học viên truy cập. - **Cảnh báo uptime** — nếu backend lỗi hoặc CSDL rớt kết nối, job sẽ đổ trên tab **Actions**, và GitHub tự gửi email cho chủ repo (`tamthientinvu-coder`) mà không cần cấu hình gì thêm.

Kiểm tra: GitHub → repo → tab **Actions** → workflow **Uptime ping** — mỗi lần chạy phải là dấu tích xanh. Có thể bấm **Run workflow** để kiểm tra ngay không cần chờ lịch.

12.4. Deploy Notifications — bật sẵn theo mặc định của từng nền tảng

Không phải một gói cần cài, mà là cấu hình thông báo có sẵn ở tài khoản Render và Vercel:

Nền tảng	Mặc định	Xem/đổi ở đâu
Render	Tự gửi email khi build hoặc deploy thất bại, gửi tới email tài khoản	Render Dashboard → góc trên phải, avatar → Account Settings → Notifications
Vercel	Tự gửi email khi deploy production thất bại	Vercel Dashboard → avatar → Settings → Notifications — có thể bật thêm Deployment Summary (tóm tắt mỗi lần deploy) hoặc nói Slack/Discord qua mục Integrations nếu muốn nhận vào kênh chat thay vì email

Vì đây là cài đặt theo tài khoản (yêu cầu đăng nhập), cha cần vào hai trang trên và bật tay một lần — không có API hay tệp cấu hình nào trong repo điều khiển được việc này.

13. Quản lý dữ liệu bằng Prisma Studio từ máy tính

Prisma Studio là giao diện web (chạy cục bộ trên máy, không phải trên Render) để xem và sửa trực tiếp dữ liệu trong PostgreSQL — tiện khi cần kiểm tra hoặc chỉnh dữ liệu thật mà không phải viết SQL.

13.1. Với cơ sở dữ liệu cục bộ (Docker, lúc lập trình)

```
cd backend
npx prisma studio
```

Tự mở `http://localhost:5555`, đọc `DATABASE_URL` sẵn có trong `backend/.env` (trở vào Postgres cổng 5433 chạy bằng Docker) — không cần cấu hình gì thêm.

13.2. Với cơ sở dữ liệu thật trên Render — cần một bước chuẩn bị

`backend/.env` (cục bộ) đang trở vào Postgres Docker, **không phải** cơ sở dữ liệu Render — Prisma Studio mặc định sẽ mở nhầm CSDL rỗng ở máy nếu chạy thẳng `npx prisma studio`. Repo đã cài sẵn `dotenv-cli` và thêm lệnh riêng để tách hai môi trường:

```
npm run studio:prod
```

Lệnh này đọc biến từ tệp `backend/.env.production.local` — **tệp này cha tự tạo, không có sẵn trong repo và không bao giờ lên git** (đã nằm trong mẫu `.env.*.local` của `.gitignore`). Các bước một lần:

1. Render Dashboard — database `learnquiz-db` — tab **Connections**
2. Copy chuỗi **External Database URL** (khác với Internal Database URL dùng ở Bước 3 — External mới kết nối được từ ngoài mạng Render, tức là từ máy cá nhân)
3. Tạo tệp mới `backend/.env.production.local` với đúng một dòng:

```
DATABASE_URL="<dán External Database URL vừa copy>?sslmode=require"
```

(thêm `?sslmode=require` ở cuối — Render yêu cầu kết nối từ ngoài phải mã hoá TLS, thiếu cờ này sẽ báo lỗi kết nối)

4. Chạy `npm run studio:prod` — Prisma Studio mở ở `http://localhost:5555`, lần này là dữ liệu **thật** trên Render

Cẩn trọng: đây là sửa trực tiếp trên dữ liệu thật đang phục vụ người dùng thật (nếu có) — sửa/xoá nhầm không có “hoàn tác”. Đóng tab và dừng lệnh (`Ctrl+C`) ngay sau khi xong việc, đừng để Prisma Studio mở treo cả buổi.